

Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого

—
Институт компьютерных наук и кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 3

«Нахождение n -ного элемента последовательности»

по дисциплине «Структуры данных»

Выполнил
студент гр.5151001/40001

<подпись>

Волошкевич М.А.

Преподаватель /
ассистент

<подпись>

Семьянов П.В.

Санкт-Петербург
2025г.

1. Цель работы.

Реализовать алгоритм для нахождения энного элемента в последовательности, состоящей из чисел, которые являются произведением степеней чисел 3, 5 и 7. Проверить корректность работы алгоритма и его эффективность.

2. Постановка задачи

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri.

3. Теоретические исследования

Для выполнения работы необходимо было изучить алгоритмы генерации последовательностей, а также методы динамического управления памятью для хранения промежуточных результатов. Основные теоретические аспекты:

- Последовательность формируется из чисел вида $3^a \times 5^b \times 7^c$, где a , b , c — неотрицательные целые числа.
- Для эффективного нахождения следующего элемента используются указатели на текущие минимальные кандидаты для умножения на 3, 5 и 7.
- Динамический массив используется для хранения элементов последовательности с возможностью его компактификации для оптимизации использования памяти.

Говно список говна:

1. Fisrt
2. Second
3. Third

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos.

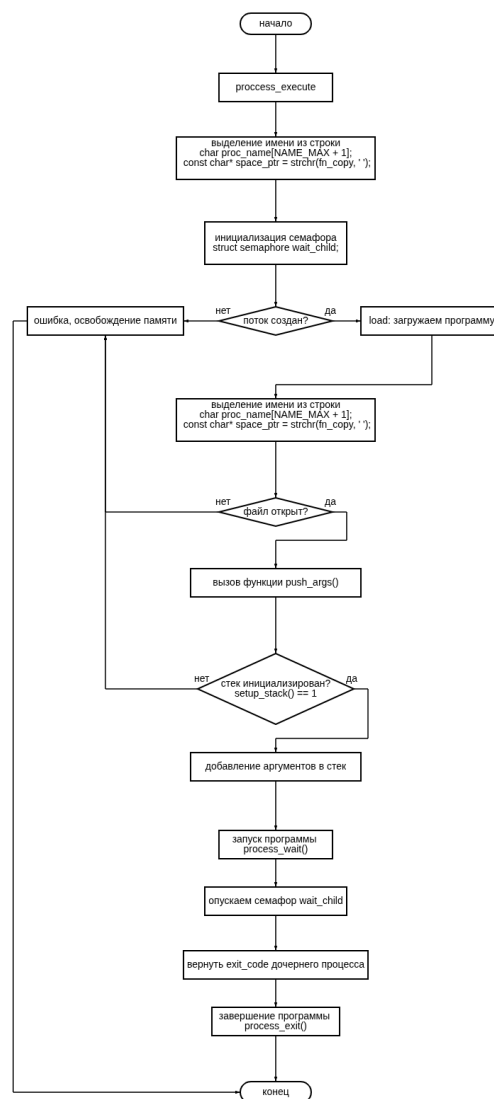


Рис. 1. Блок схема работы программы

В Рис. 1 у нас находится блок схема с говном

Amount	Ingredient
360g	Baking flour
250g	Butter (room temp.)
150g	Brown sugar
100g	Cane sugar
100g	70% cocoa chocolate
100g	35-40% cocoa chocolate
2	Eggs
Pinch	Salt
Drizzle	Vanilla extract

Таблица 1. Таблица с говном

В Таблица 1 какая-то залуца

Приложения

main.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "qr_encoder.h"
#include "lzw.h"

void print_usage()
{
    printf("Usage:\n");
    printf("  Compression:  ./main -c <input_file> <output_file> --
password "
        "<password>\n");
    printf("  Decompression: ./main -d <input_file> <output_file> --
password "
        "<password>\n");
}

int main(int argc, char* argv[])
{
    if (argc != 6 || strcmp(argv[4], "--password") != 0)
    {
```

```

    print_usage();
    return 1;
}

char mode = argv[1][1];
char* input_file = argv[2];
char* output_file = argv[3];
char* password = argv[5];

if (mode == 'c')
{
    // Читаем входной файл
    FILE* f = fopen(input_file, "rb");
    if (!f)
    {
        perror("input");
        return 1;
    }
    fseek(f, 0, SEEK_END);
    long size = ftell(f);
    fseek(f, 0, SEEK_SET);
    char* buf = malloc(size + 1); // +1 для '\0'
    fread(buf, 1, size, f);
    buf[size] = '\0';
    fclose(f);

    // Генерируем QR PPM в памяти
    unsigned char* ppm_buf = NULL;
    size_t ppm_size = 0;
    if (!generate_qr_mem(buf, size, password, &ppm_buf,
&ppm_size))
    {
        printf("QR generation failed\n");
        free(buf);
        return 1;
    }
    free(buf);

    // Сжимаем LZW в память
    unsigned char* compressed = NULL;

```

```

size_t compressed_size =
    lzw_compress_mem(ppm_buf, ppm_size, &compressed);
free(ppm_buf);

// Сохраняем результат
FILE* out = fopen(output_file, "wb");
fwrite(compressed, 1, compressed_size, out);
fclose(out);
free(compressed);

printf("Compression completed. Output: %s\n", output_file);
}
else if (mode == 'd')
{
    // Читаем сжатый файл
    FILE* f = fopen(input_file, "rb");
    if (!f)
    {
        perror("input");
        return 1;
    }
    fseek(f, 0, SEEK_END);
    long size = ftell(f);
    fseek(f, 0, SEEK_SET);
    unsigned char* compressed = malloc(size);
    fread(compressed, 1, size, f);
    fclose(f);

    // Декомпрессия в память
    unsigned char* decompressed = NULL;
    size_t decompressed_size =
        lzw_decompress_mem(compressed, size, &decompressed);
    free(compressed);

    // Сохраняем декомпрессированный PPM
    FILE* out = fopen(output_file, "wb");
    fwrite(decompressed, 1, decompressed_size, out);
    fclose(out);
    free(decompressed);
}

```

```
    printf("Decompression completed. Output: %s\n", output_file);  
}  
else  
{  
    print_usage();  
    return 1;  
}  
return 0;  
}
```